

KAFKA ON KUBERNETES • WITH DAPR

# Kafka, on *Kubernetes*.

Six Kafka ideas. One live swap from Redis.  
Dapr as the contract — so your app never notices.

DAPR 1.17 • STRIMZI 0.51 • K3S

# By the end, you can *swap a broker* without touching app code.

01

## Component model

Why apps don't name their backing services — and how that becomes portability.

02

## The swap

Redis to Kafka by editing one YAML and rolling two deployments.

03

## Semantic leak

Where Kafka's consumer-group rules quietly differ from Redis fan-out.

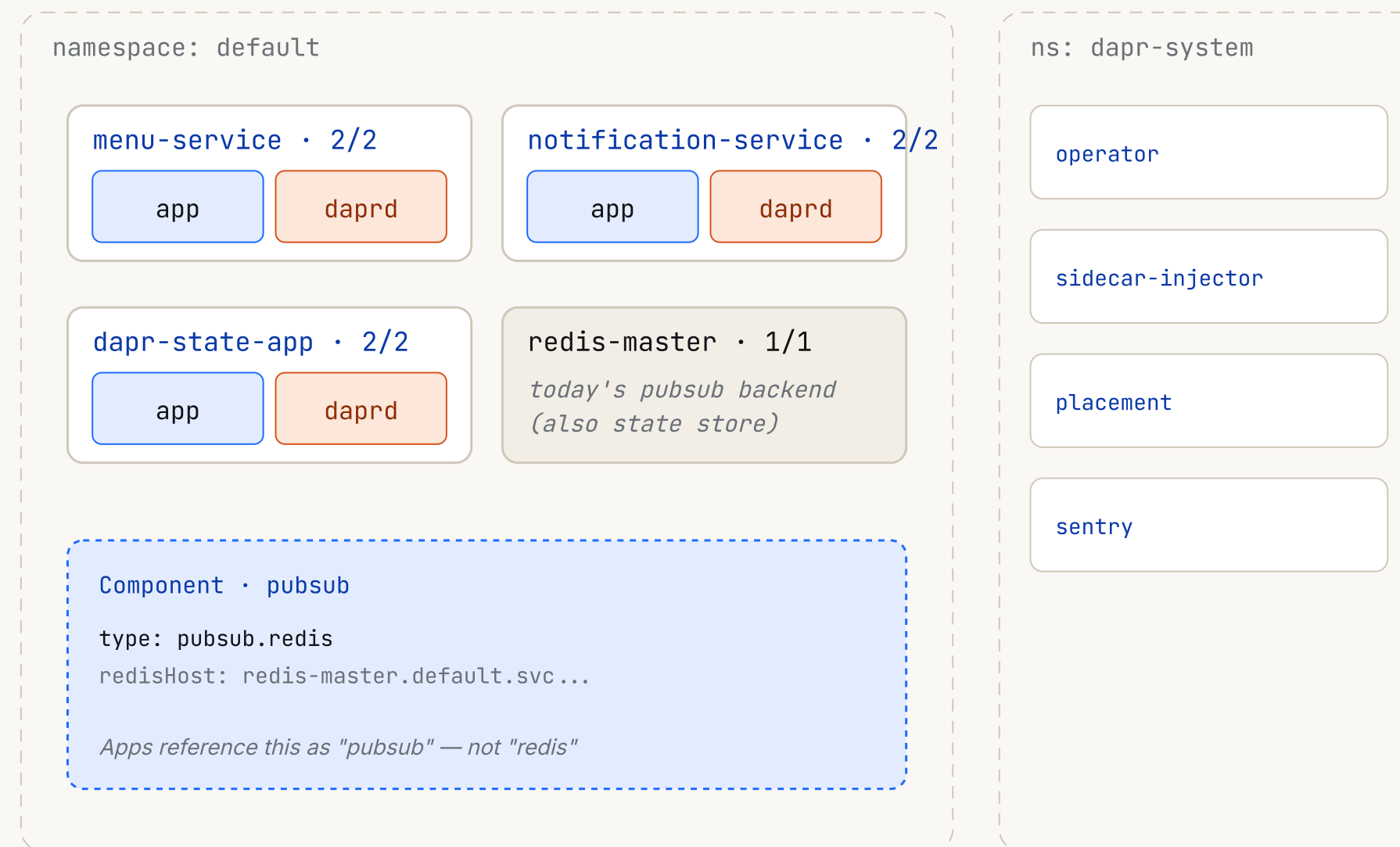
04 ★

## The bug you'll cause

consumerGroup, one line, kills fan-out. Today's most useful lesson.

# Where we walked in.

*2/2* means app + sidecar.



Every app pod has **two containers**:  
your code + a Dapr sidecar.

The sidecar *is* Dapr — for that pod. Component CRs  
configure all sidecars in a namespace.

```
$ kubectl get component pubsub -o jsonpath='{.spec.type}'  
pubsub.redis
```

PRIMER • KAFKA IN 10 MINUTES

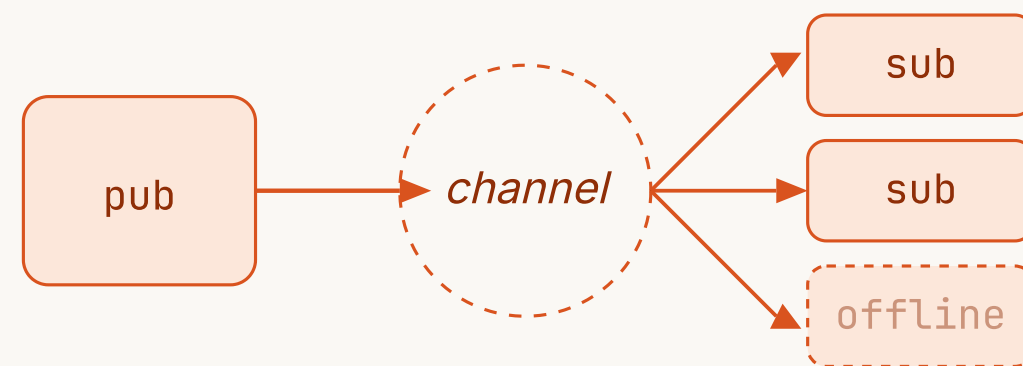
# Six ideas. *Then we swap.*

You don't need Kafka mastery today. You need just enough model to understand what the sidecar is doing — and to recognise one very specific gotcha when it bites.

# Pub/sub is an *API shape*. Not a storage model.

## REDIS PUB/SUB

Shouts into  
a *room*.



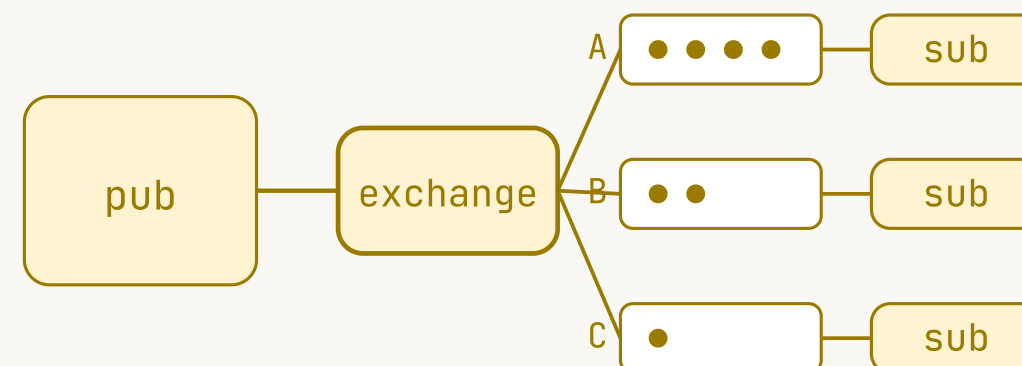
*offline sub → message lost*

If nobody's listening, the message is gone.

No persistence • No replay • Fan-out is free

## RABBITMQ

Hands you  
a *letter*.



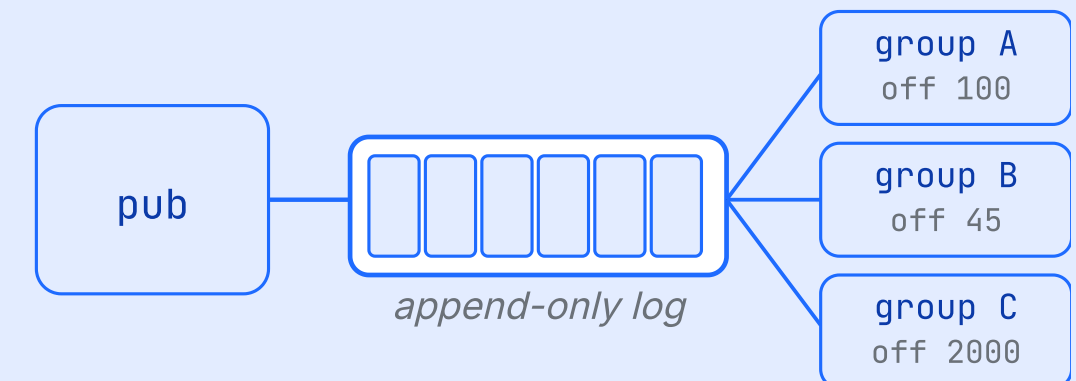
*one copy per queue • held until ack*

Broker tracks delivery. Ack, then it's deleted.

Durable until ack • No replay • Rich routing

## KAFKA • TODAY

Writes to an  
*archive*.



*one copy stored • each group has its own bookmark*

**Consumer tracks its own offset.**

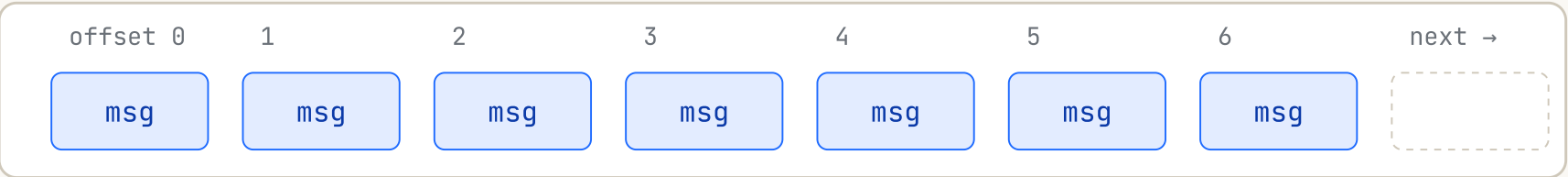
Anyone can replay.

Durable until retention • Replayable •  
Fan-out via groups



A *topic* is a named, append-only log.

topic:task-events



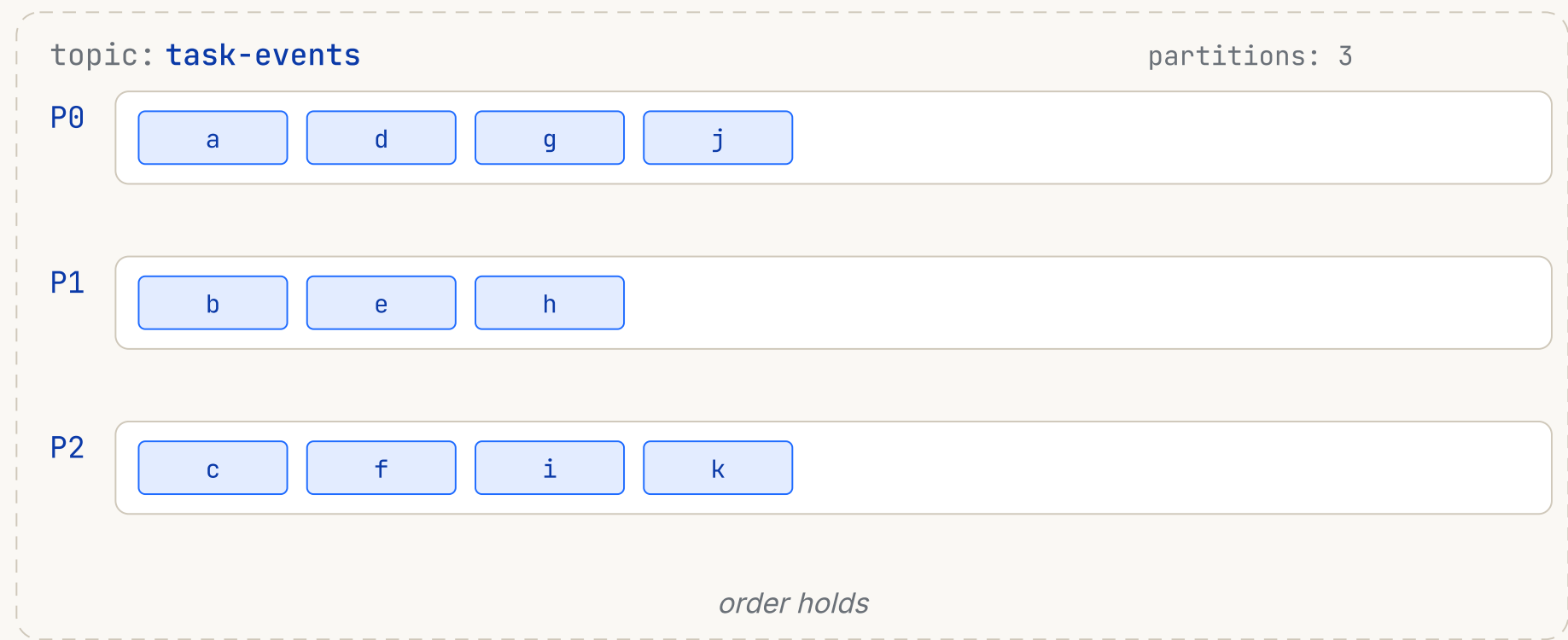
writes only go to the end →

reads can start from any offset — past messages stay

A log file you can **write to** and **read from** — but never edit or shrink arbitrarily.

Today's topic: `task-events`. Every *task-created* event lands here.

# One topic, *split into N ordered logs.*



a partition · no global order

*within*

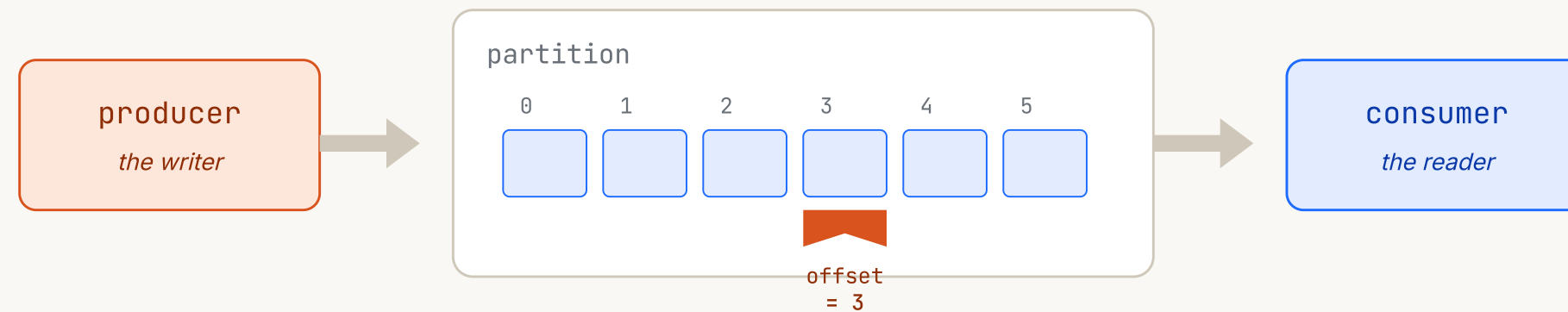
*across*

**Why** — partitions are how Kafka parallelises: more partitions = more consumers can read in parallel.

partitions

Producer picks partition by `hash(key)` . Same key

# Writers write. Readers *bookmark*.



*the offset is the consumer's bookmark — move it back to replay, forward to skip*

**Producer** — writes. Picks partition by `hash(key)`, or round-robin.

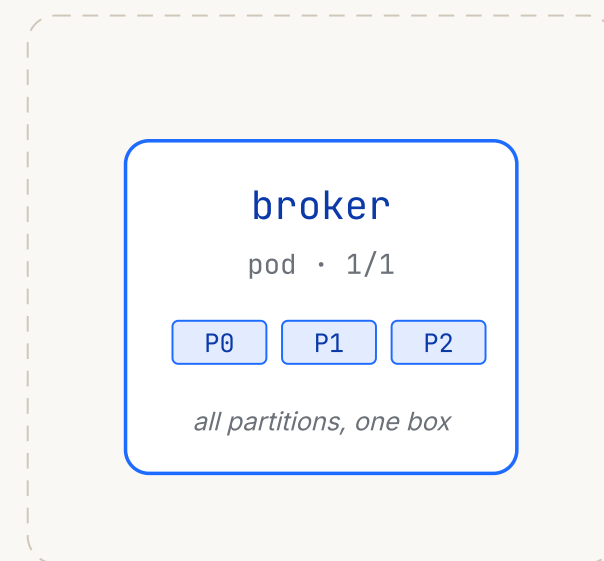
**Consumer** — reads. Tracks its own position.

Kill it, restart it → resumes from the last committed offset. Give it a new identity → replays from the beginning.

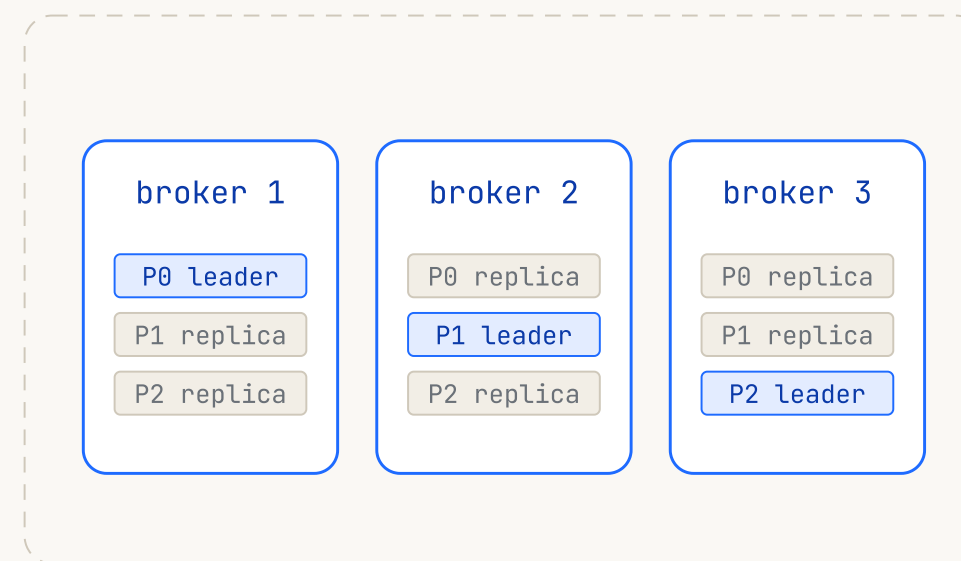


A *broker* is a pod.  
A cluster is a team of brokers.

today — demo



production — durable



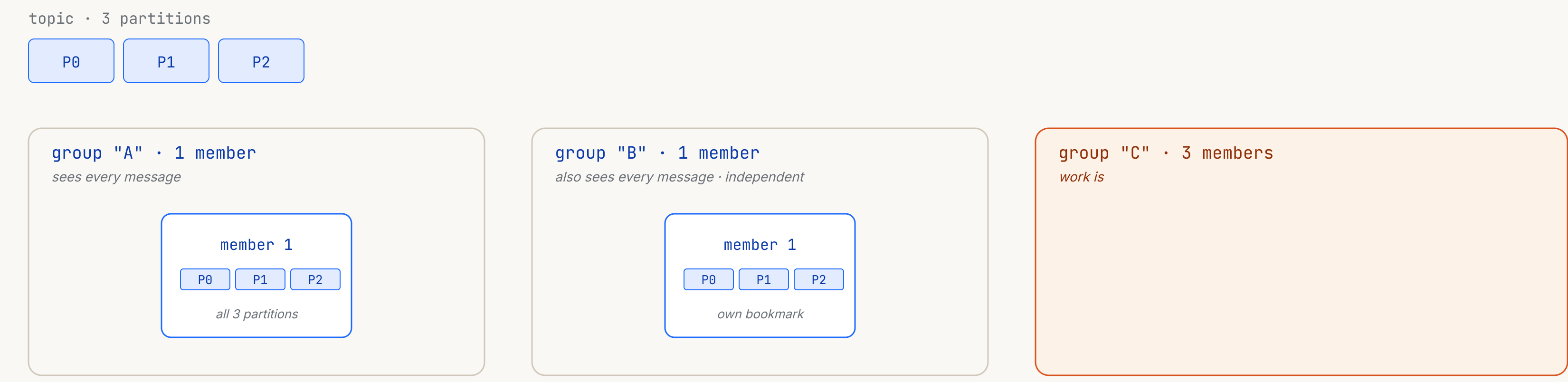
**Broker** — a server that holds partition data.

**Cluster** — several brokers working together.

Today: **1 broker**. Prod: **3+** so a lost node doesn't lose data.

**KRaft** = no ZooKeeper. Simpler. That's all we need to say today.

A *consumer group* is a label that says:  
*"we're a team sharing the work."*



*split*

— each message goes to exactly one

member 1 P0 member 2 P1 member 3 P2

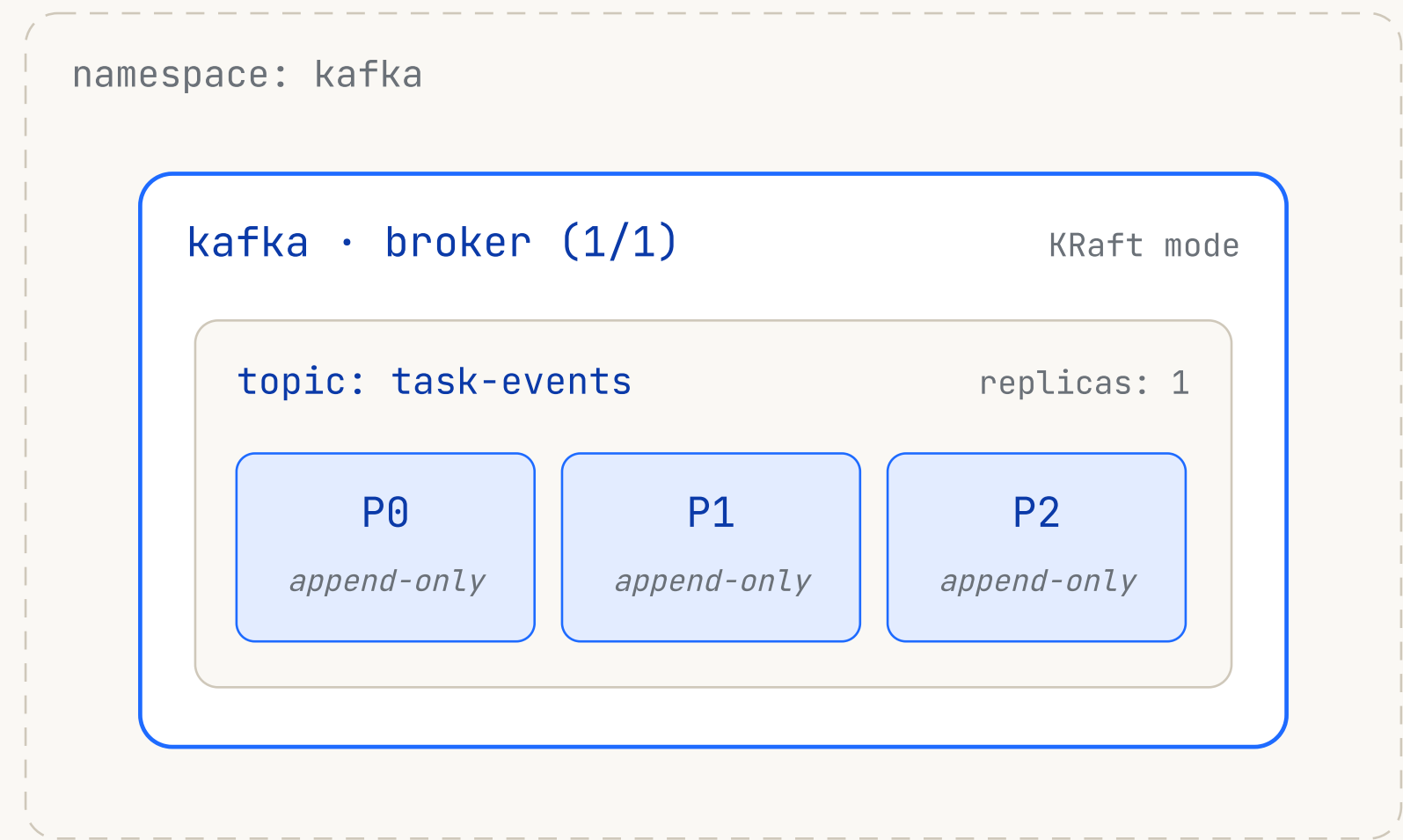
different groups → fan-out · same group, many members → work queue

# Redis *broadcasts.* Kafka *remembers.*

|                 | REDIS PUB/SUB             | KAFKA                       |
|-----------------|---------------------------|-----------------------------|
| Persistence     | ✗ none · forward & forget | ✓ durable log on disk       |
| Replay          | ✗ impossible              | ✓ via offset                |
| Fan-out default | ✓ automatic               | ⚠ depends on consumer group |
| Ordering        | per channel               | per partition               |
| Load balancing  | ✗ not built in            | ✓ via consumer groups       |

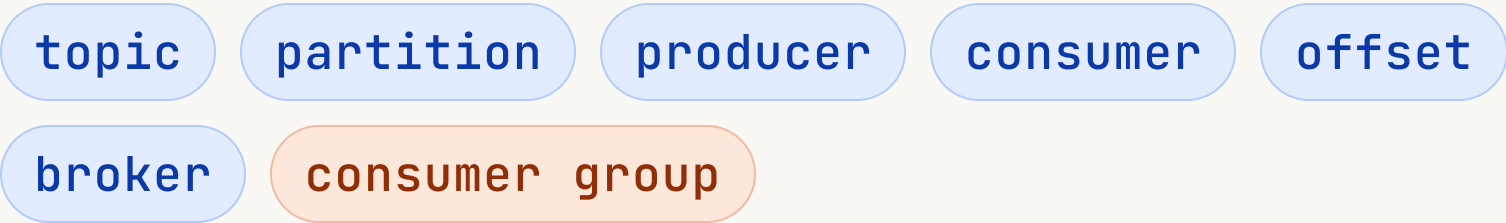
*"That's the gap today closes – and the gap that makes consumer groups matter."*

# One broker. Three partitions. *task-events*. Let's go.



Your model is sufficient. Two services will each become their own group. Each will see every event.

Listen for the moment I add `consumerGroup` to the Component — and predict what breaks.



SECTION · I

A Component is  
an *abstraction*  
over  
infrastructure.

Apps name capabilities. Components name implementations. The wire between them is a string.



# Everything you'll run today lives in *one repo*.

```
$ cd ~/hello-dapr
```

## 1 • the wire

### Dapr Component

pubsub.yaml

One file. One `type:` line. This is the thing that flips.

## 2 • the broker

### Kafka stack

kafka-nodepool.yaml

kafka-cluster.yaml

kafka-topic.yaml

Strimzi CRDs. Lives in the `kafka` namespace.

## 3 • untouched

### App code

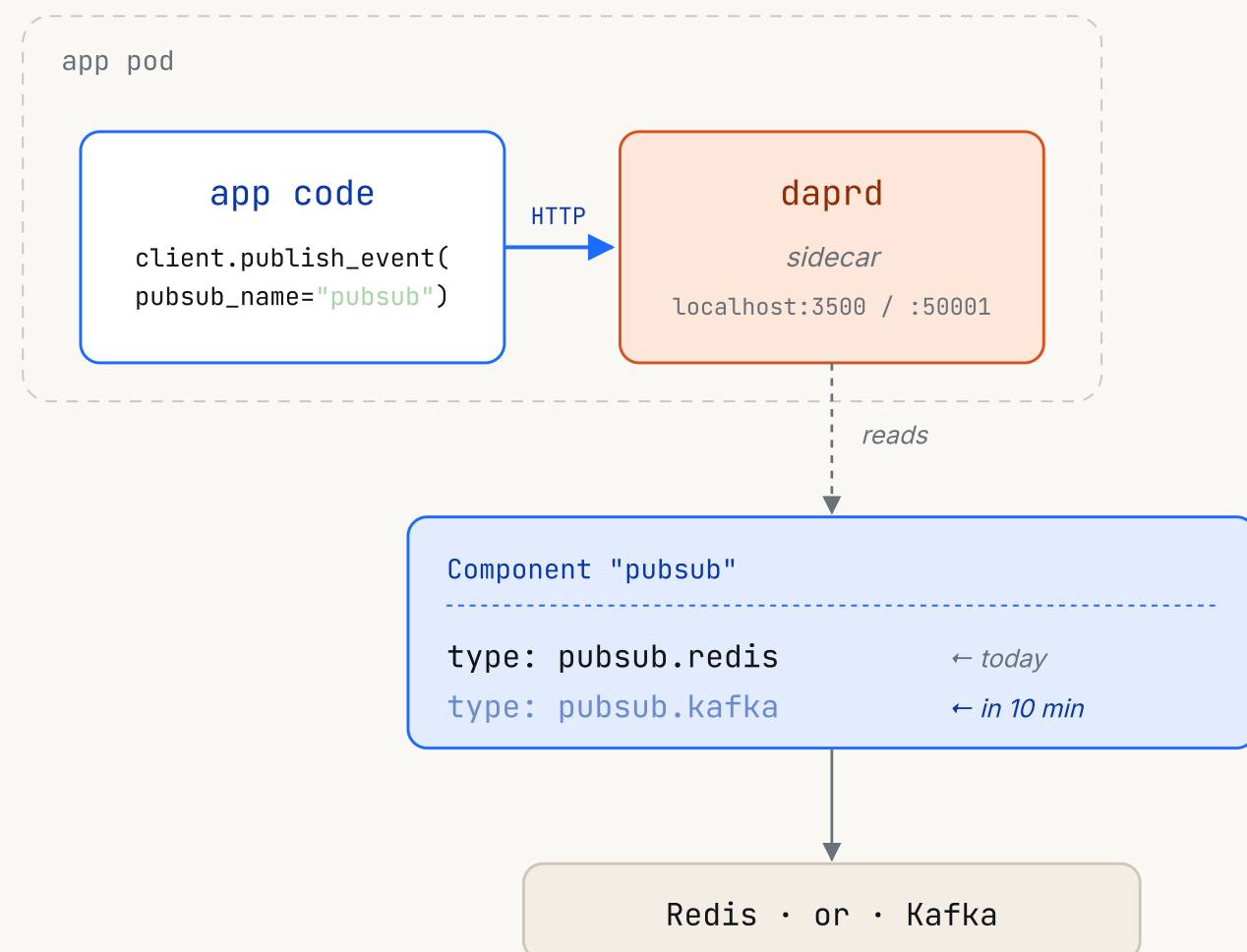
menu-service/

notification-service/

You won't open these today. *That's the point.*

If you get lost during the demo – come back to this slide.

# Your code calls *"pubsub"*. What that means is YAML, not Python.



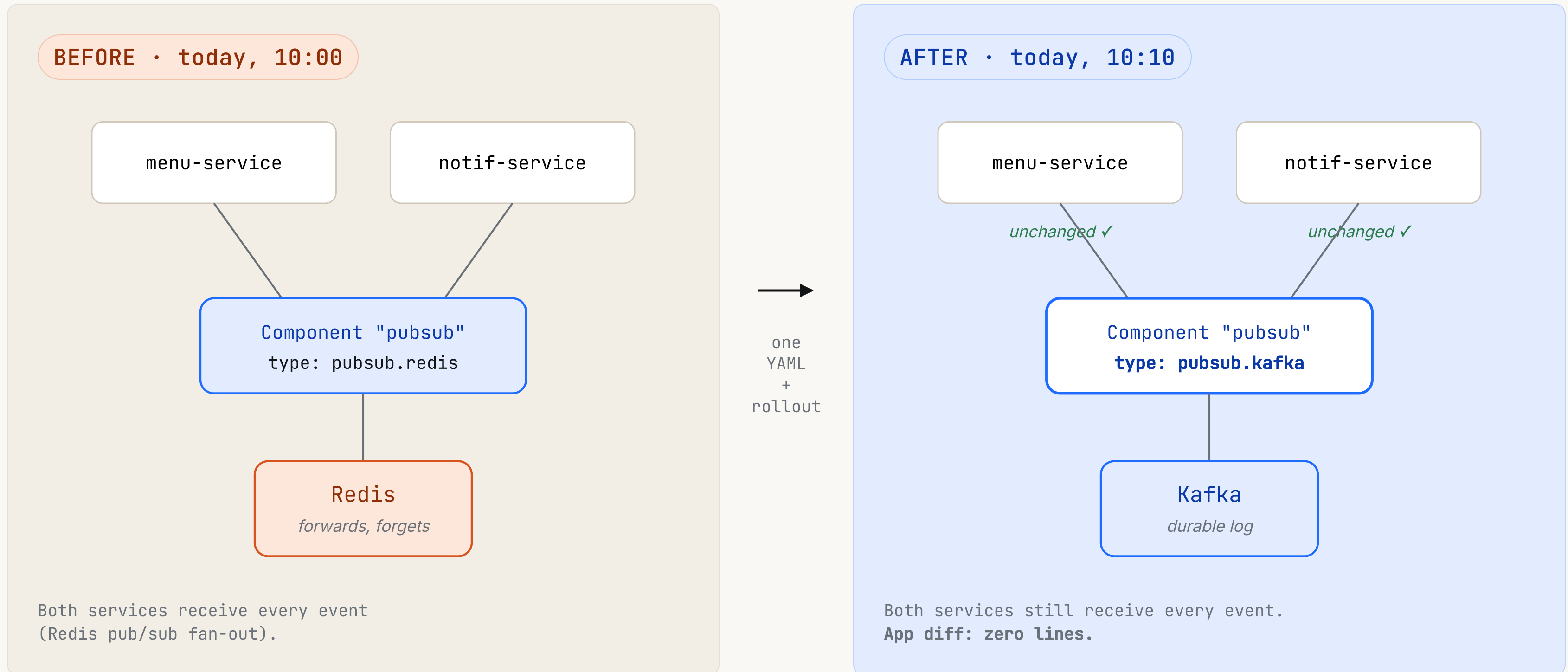
The string `"pubsub"` is the **logical name of a Component**.

The Component decides what that name *means*.

12-factor backing services, but materialised — for once — as a real Kubernetes resource you can

`kubectl edit`.

# Same diagram. *Bottom box swaps.*





# The API matches. The *guarantees* don't.

| ASPECT         | REDIS PUB/SUB                               | KAFKA                                                |
|----------------|---------------------------------------------|------------------------------------------------------|
| Durability     | None — no subscriber, no message.           | Durable log, retained by <code>retention.ms</code> . |
| Replay         | Impossible.                                 | <code>--from-beginning</code> reads history.         |
| Fan-out ★      | Every subscriber gets every message — free. | <b>Depends on consumer groups.</b> ← will matter     |
| Ordering       | Per channel.                                | Per partition.                                       |
| Load balancing | Not built in.                               | Consumer group = work queue.                         |

# Pre-class check.

## *Do this before Hands On.*

```
# Apps healthy, on Redis, no Kafka residue
$ kubectl get pods -n default
NAME                                READY   STATUS
menu-service-xxx                    2/2     Running
notification-service-xxx            2/2     Running
redis-master-0                      1/1     Running

$ kubectl get component pubsub -n default \
  -o jsonpath='{.spec.type}'
pubsub.redis

$ kubectl get crd | grep kafka.strimzi.io
# expect empty – no leftovers from a prior class

$ helm repo list | grep strimzi
strimzi    https://strimzi.io/charts/
```

SECTION · II

# Stand up Kafka. (*Recap* from last class.)

Three apply commands. Ninety seconds wall-clock. One naming gotcha you must call out.

# Operator → nodepool → cluster → topic. *Four applies.*

```
# 1 – operator
$ kubectl create namespace kafka
$ helm repo add strimzi https://strimzi.io/charts/
$ helm install strimzi-kafka-operator \
  strimzi/strimzi-kafka-operator -n kafka

# 2 – the machines (nodepool)
$ kubectl apply -f kafka-nodepool.yaml

# 3 – the cluster config
$ kubectl apply -f kafka-cluster.yaml
$ kubectl wait kafka/task-events -n kafka \
  --for=condition=Ready --timeout=5m

# 4 – pre-create the topic
$ kubectl apply -f kafka-topic.yaml

$ kubectl get kafkatopic -n kafka
NAME          READY   PARTITIONS
task-events   True    3
```

1. Operator first.  
A controller watching for `Kafka` CRs.
2. `KafkaNodePool` = the machines.  
1 replica, dual-role (controller + broker), ephemeral storage.
3. `Kafka` CR named `task-events`.  
KRaft mode, Kafka 4.1.1. Name becomes bootstrap prefix.
4. `KafkaTopic task-events`, 3 partitions.  
Pre-created so we own the partition count.

# Why is it *task-events-kafka-bootstrap*?

*CR name (you chose)*

*literal suffix Strimzi adds*

`kafka-kafka-bootstrap.kafka.svc.cluster.local:9092`

*namespace*

## WHY IT HAPPENS

Strimzi generates a Service named

`<cr-name>-kafka-bootstrap`.

You named the CR `kafka`. So:

`task-events-kafka-bootstrap`. Yes, kafka twice.

## FAILURE MODE

Tutorials sometimes say `kafka-bootstrap`. The sidecar will log *"no such host"* and silently never connect. **Most common failure of the day.**



SECTION · III

# The swap. *One YAML.*

From here on, everything fits on one screen at a time. Ten minutes of demo. We'll pause at each command for you to predict what happens next.

# Here's the current Component.

## *What happens if I change one line?*

```
$ cat ~/hello-dapr/pubsub.yaml

apiVersion: dapr.io/v1alpha1
kind: Component
metadata:
  name: pubsub
  namespace: default
spec:
  type: pubsub.redis
  version: v1
  metadata:
    - name: redisHost
      value: redis-master.default.svc.cluster.local:6379
```

SAY

*If I flip `type: pubsub.redis` to `pubsub.kafka` and run `kubectl apply` – what happens?*

EXPECTED ANSWERS

**"It breaks."** → Why? The app doesn't name Redis anywhere.

**"It just works."** → What has to happen for that to be true?

**"Sidecars need to reload."** → Exactly. ★

# The whole class, in *five lines of diff*.

```
kind: Component
metadata:
  name: pubsub          # ← do NOT change the name
spec:
- type: pubsub.redis
+ type: pubsub.kafka
  version: v1
  metadata:
-   - name: redisHost
-     value: redis-master.default.svc.cluster.local:6379
+   - name: brokers
+     value: task-events-kafka-bootstrap.kafka.svc.cluster.local:9092
+   - name: authType
+     value: none
  # NO consumerGroup — see slide 21
```

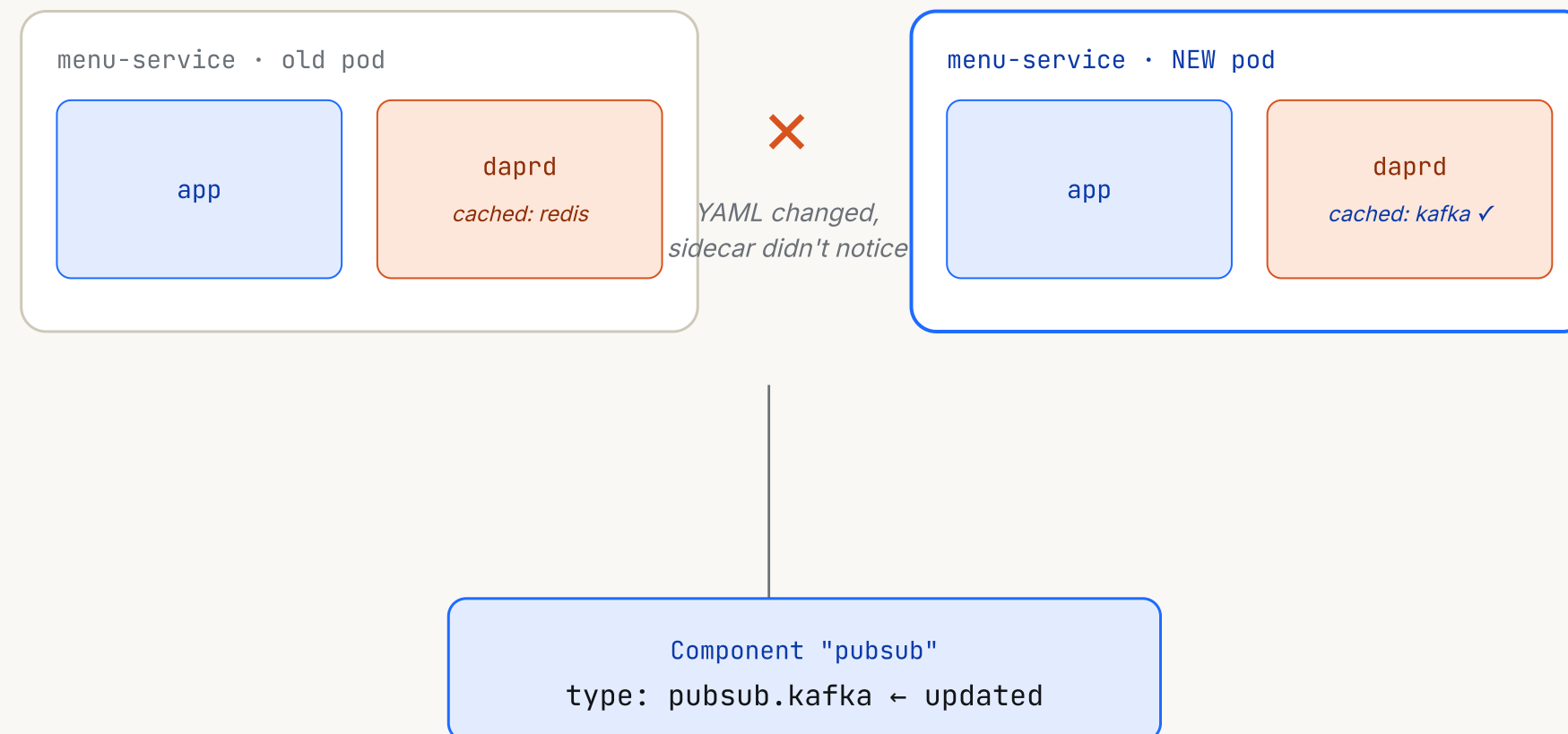
```
$ kubectl apply -f pubsub.yaml
component.dapr.io/pubsub configured
```

Note **configured**, not *created*.  
The name stayed. The implementation changed.

Don't add `consumerGroup` here.  
We'll induce that bug on purpose later.



# Sidecars cache Components *at boot*. YAMLs aren't always live.



```
$ kubectl rollout restart \
  deploy/menu-service \
  deploy/notification-service \
  -n default

$ kubectl rollout status \
  deploy/menu-service -n default
deployment "menu-service" successfully rolled out
```

Each deployment gets a new ReplicaSet. New pods spawn. New sidecars boot. New sidecars read the new Component CR.

Look for one line.  
*"Component loaded: pubsub  
(pubsub.kafka/v1)"*

```
$ kubectl logs deploy/menu-service \
  -c daprd -n default | grep pubsub

INFO Component loaded: pubsub (pubsub.kafka/v1)
INFO Initialised pubsub component pubsub
INFO Consumer group rebalance strategy
    set to 'range'
INFO Connected to Kafka: task-events-kafka-bootstrap...
```

GREEN-PATH SIGNALS

- ✓ *Component loaded*
- ✓ *Consumer group rebalance strategy set*
- ✓ *No "failed to connect" lines*

IF YOU SEE...

no such host in daprd logs

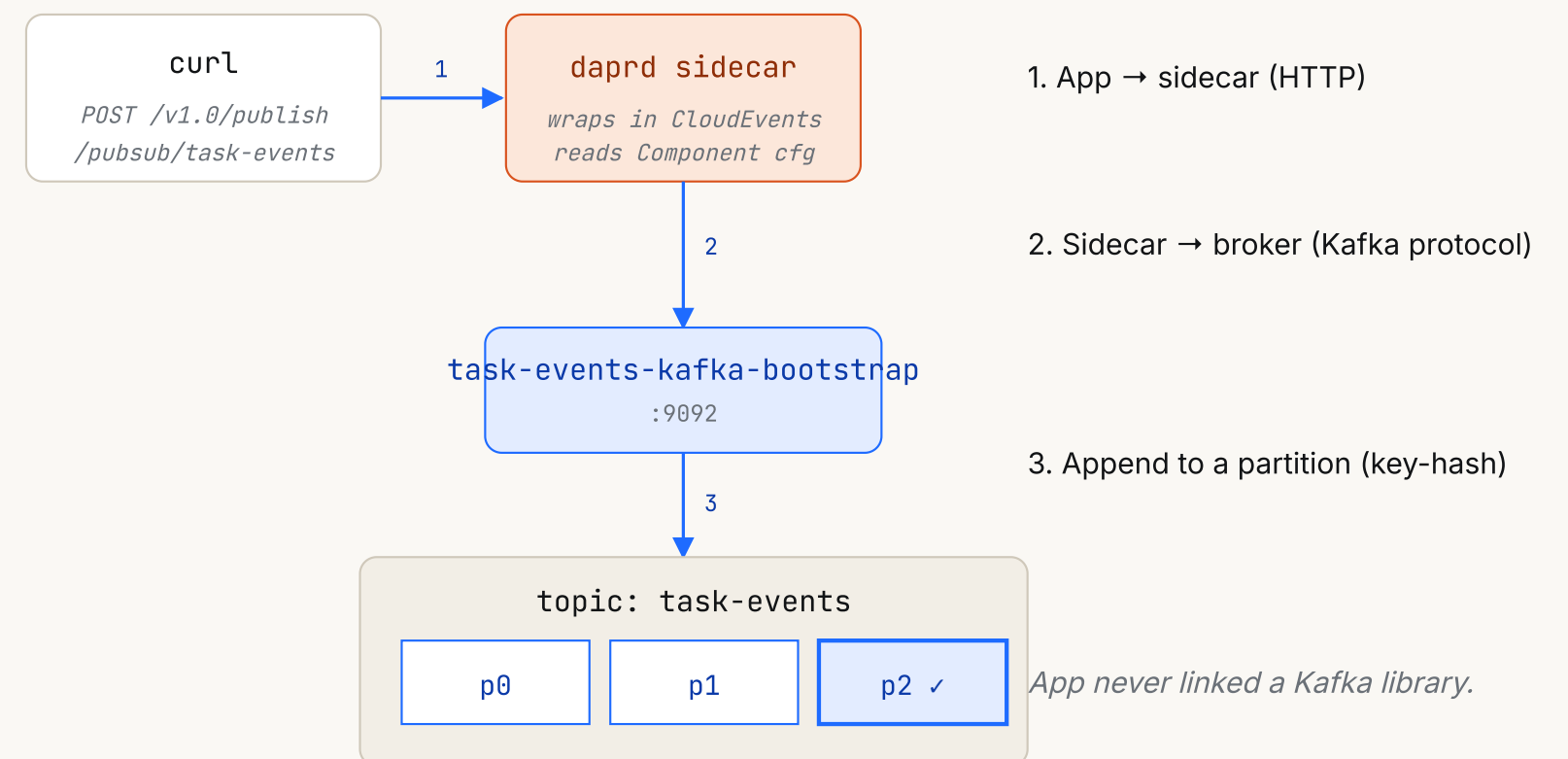
→ go back, check task-events-kafka-bootstrap .

Plain HTTP. *No Kafka client.*  
The sidecar does the rest.

```
$ kubectl port-forward -n default \
  deploy/menu-service 3500:3500 &
$ sleep 3

$ curl -X POST \
  http://localhost:3500/v1.0/publish/pubsub/task-events \
  -H "Content-Type: application/json" \
  -d '{"event_type":"task.created",
      "task_id":"demo-1",
      "title":"kafka-via-dapr"}'

# HTTP/1.1 204 No Content
```



Both services log the event.  
*It looks just like Redis.*

menu-service • app logs

```
$ kubectl logs deploy/menu-service \
  -c app -n default --tail=5

INFO:main:MENU EVENT RECEIVED:
  {'data': {
    'event_type': 'task.created',
    'task_id': 'demo-1',
    'title': 'kafka-via-dapr'
  }, 'topic': 'task-events' }
```

notification-service • app logs

```
$ kubectl logs deploy/notification-service \
  -c app -n default --tail=5

INFO:main:EVENT RECEIVED:
  {'data': {
    'event_type': 'task.created',
    'task_id': 'demo-1',
    'title': 'kafka-via-dapr'
  }, 'topic': 'task-events' }
```

**Why does fan-out still work?** Dapr defaults each app's `consumerGroup` to its `app-id`. Two services = two groups = both see every message. *Hold this thought for slide 21.*

Your app sent *title*.  
Kafka is holding a *CloudEvents envelope*.

```
$ kubectl run peek -n kafka --rm -it \
  --image=quay.io/strimzi/kafka:0.51.0-kafka-4.1.1 \
  --restart=Never -- \
  bin/kafka-console-consumer.sh \
  --bootstrap-server task-events-kafka-bootstrap:9092 \
  --topic task-events \
  --from-beginning --timeout-ms 10000
```

A throwaway Kafka client pod, talking directly to the broker.  
**The app didn't know we did this.**

#### WHAT'S ACTUALLY ON THE WIRE

```
{
  "specversion": "1.0",           // CloudEvents
  "id": "840d5da7-...",
  "source": "menu-service",
  "type": "com.dapr.event.sent",
  "time": "2026-04-18T13:48:00Z",
  "topic": "task-events",
  "pubsubname": "pubsub",
  "datacontenttype": "application/json",
  "data": {                       // ← your payload
    "event_type": "task.created",
    "task_id": "demo-1",
    "title": "kafka-via-dapr"
  }
}
```



SECTION · IV · THE GOTCHA

One word.  
*consumerGroup*.  
Fan-out gone.

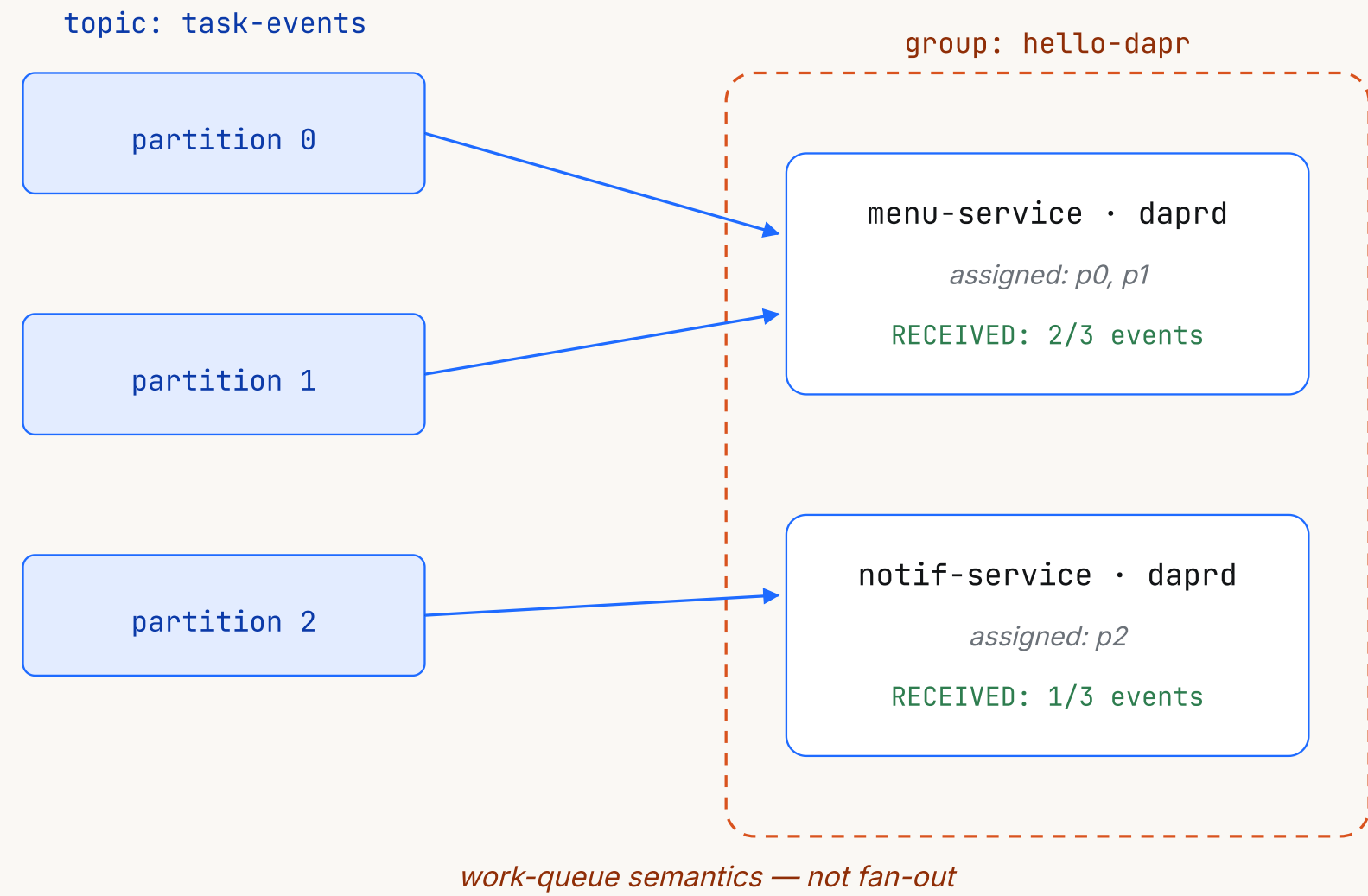
The class's best moment. We're going to break it on purpose so you recognise it in the wild.

# Add one line. Re-apply.

*Now only one service receives.*

```
spec:
  type: pubsub.kafka
  metadata:
    - name: brokers
      value: task-events-kafka-bootstrap...
+   - name: consumerGroup
+     value: hello-dapr
```

Both sidecars now join the *same* Kafka group. Kafka assigns the topic's 3 partitions across them — each message hits one partition, so exactly one group member consumes it.



# Same topic. Three patterns.

## *Three different Components.*

| PATTERN                  | HOW TO CONFIGURE                                                                               | USE WHEN                                                           |
|--------------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| Fan-out broadcast        | Omit <code>consumerGroup</code> → Dapr defaults to <code>app-id</code> . Each app = own group. | Every subscriber needs every event — notifications, audit, caches. |
| Load-balanced work queue | Set the <b>same</b> <code>consumerGroup</code> across replicas.                                | One job per event across a pool of workers.                        |
| Replay from origin       | Add <code>initialOffset: oldest</code> in metadata.                                            | A new subscriber wants history, not just live traffic.             |

→ The same physical Kafka topic can serve all three patterns simultaneously, each via a different *Component* definition.



# What you just saw.

- 01 One YAML apply + one rollout = new broker. Code unchanged.
- 02 Sidecars cache Components at boot. Changes need a rollout to take effect.
- 03 Kafka + Dapr default = fan-out, because Dapr uses `app-id` as the consumer group.
- 04 Shared `consumerGroup` = work-queue semantics. Useful — but opt in.
- 05 CloudEvents envelope is what's actually on the wire — your code sends payload, Dapr wraps metadata.

SAY

*Dapr's abstraction is real — but the underlying broker's semantics leak through. Knowing both layers makes you dangerous.*

# When it doesn't work, it's *almost always* one of these six.

Q.

DNS: no such host kafka-bootstrap

Tutorial hostname is wrong — missing the cluster-name prefix. Use `task-events-kafka-bootstrap.kafka.svc.cluster.local`.

Q.

Publish returns 204 but nothing in app logs

Sidecar didn't pick up the new Component. `kubectl rollout restart` the deployments.

Q.

Only one service receives

Shared `consumerGroup` in the Component — omit it and you're back to fan-out.

Q.

Component pubsub not found

Namespace mismatch — Component lives in `default`, app in another. Match them.

Q.

KafkaTopic stuck at READY: False

Missing `entityOperator.topicOperator: {}` in the Kafka CR. Add it; reconcile.

Q.

v1beta2 deprecation warnings on apply

Harmless. v1 is the stable API now; the operator still accepts the old one for compatibility.

Same Lesson  
next week.  
*Same starting  
state.*

```
# back to Redis
git -C ~/hello-dapr checkout pubsub.yaml
kubectl apply -f ~/hello-dapr/pubsub.yaml
kubectl rollout restart deploy/menu-service \
  deploy/notification-service -n default

# tear down Kafka
kubectl delete kafkatopic,kafka,kafkanodepool \
  --all -n kafka --wait=false
helm uninstall strimzi-kafka-operator -n kafka
kubectl delete namespace kafka --wait
kubectl get crd -o name | grep kafka.strimzi.io \
  | xargs -r kubectl delete
```

End state = identical to pre-class.  
Repeatable demo.

QUESTIONS?

BREAK • 10 MIN